



МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Операционные системы и архитектура компьютера»

на тему: «Каналы и процессы»

Выполнил:

студент гр. М8О-205БВ-25

Матков Д.А.

Проверил: Филимонов Н.С.

Оценка:

Дата:

Цель работы: приобретение практических навыков в управлении процессами ОС и обеспечении обмена данных между процессами посредством каналов.

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы

Вариант: №15. Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем назапись. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса, иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в стандартный поток вывода. Правило проверки: строка должна начинаться с заглавной буквы.

Общее описание решения:

В качестве системы для разработки был выбран Linux (EndeavourOS).

Проект состоит из 3 файлов:

1. `program.c` – точка входа. Родительский процесс, создает `fork` для запуска дочернего процесса. Получает ввод от пользователя, отправляет его по каналу в дочерний процесс, выводит ошибки, отправленные дочерним процессом.
2. `child.c` – дочерний процесс. Принимает ввод от главного процесса. Проверяет валидность. Если строка начинается с заглавной буквы, то выводит её в файл. Отправляет по каналу в родительский процесс сообщение “Ok”. Сообщение скрытое, не видно пользователю. В случае, если первый символ не является заглавной буквой, либо полученная строка пустая, в файл ничего не записывается, а в родительский процесс по каналу отправляется текст ошибки, который на стороне родительского процесса выводится в консоль пользователю.
3. `CMakeLists.txt` – файл сборки Cmake. Собирает 2 исполняемых файла: `program` и `child`.

Ключевые фрагменты кода:

Главная функция дочерней программы ():

```
int main(int argc, char* argv[]){
    if(argc < 2){
        fprintf(stderr, "Cild error: missing argument\n");
        return 1;
    }

    char buffer[1024];
    int childToParent1 = atoi(argv[1]);

    while(fgets(buffer, sizeof(buffer), stdin) != NULL){

        size_t length = strlen(buffer);

        if(isupper(buffer[0])){
            fprintf(stdout, "%s", buffer);
            dprintf(childToParent1, "\n");
        }

        else{
```

```

        if(buffer[0] == '\n') dprintf(
            childToParent1,
            "input is empty\n"
        );

        else dprintf(
            childToParent1,
            "input starts with '%c' - not uppercase\n",
            buffer[0]
        );
    }
}
close(childToParent1);
return 0;
}

```

Создание канала с проверкой на ошибку:

```

void CreatePipe(int newPipe[2]){
    if(pipe(newPipe) == -1){
        perror("Pipe creating error");
        exit(1);
    }
}

```

Запрос имени файла для вывода, открытие/создание, проверка ошибок и возврат дескриптора:

```

int OpenFile(){
    char outputFile[256];

    printf("Enter output file name: ");

    if(fgets(outputFile, sizeof(outputFile), stdin) != NULL){
        size_t length = strlen(outputFile);

        if (length > 0 && outputFile[length - 1] == '\n') {
            outputFile[length - 1] = '\0';
        }
    }
    else if(ferror(stdin)){
        perror("File name reading error");
    }

    if(outputFile[0] == '\0'){
        fprintf(stderr, "Error: file name is empty\n");
        exit(1);
    }
}

```

```

}

int outputFileId = open(
    outputFile,
    O_WRONLY | O_CREAT | O_TRUNC,
    S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH
);
if(outputFileId == -1){
    perror("File opening error");
    exit(1);
}
return outputFileId;
}

```

Дублирование одного дескриптора в другой с проверкой ошибок:

```

void DuplicateTo(int oldFd, int newFd){
    if(dup2(oldFd, newFd) == -1){
        perror("dup2 error");
        exit(1);
    }
}

```

Цикл главного процесса для получения ввода пользователя и обмена данными с дочерним процессом:

```

while(1){
    printf("Enter text: ");

    if(fgets(inputBuffer, sizeof(inputBuffer), stdin) == NULL) break;
    if(strcmp(inputBuffer, "/esc\n") == 0) break;

    dprintf(parentToChild[1], "%s", inputBuffer);

    ssize_t bytesRead = read(childToParent[0], outputBuffer, sizeof(outputBuffer) - 1);

    if(bytesRead > 0){
        outputBuffer[bytesRead] = '\0';
        if(outputBuffer[0] != '\n')
            printf("Validation error: %s", outputBuffer);
    }
}

```

```

else if(bytesRead == 0){
    fprintf(stderr, "Child process finished suddenly\n");
    status = 1;
    break;
}
else{
    perror("Read from child error");
    status = 1;
    break;
}

```

Код дочернего процесса после fork (настройка каналов, запуск дочерней программы):

```

if(processId == 0) { //child process
    DuplicateTo(parentToChild[0], STDIN_FILENO);
    DuplicateTo(outputFileId, STDOUT_FILENO);

    close(parentToChild[0]);
    close(parentToChild[1]);

    close(childToParent[0]);
    close(outputFileId);

    char fdString[16];
    sprintf(fdString, "%d", childToParent[1]);

    execl("./child", "child", fdString, (char*)NULL);

    perror("Child execution error");
    exit(1);
}

```

Запуск программы:

Консоль (ввод, вывод):

```
Enter output file name: out.txt
Enter text: Good text
Enter text:
Validation error: input is empty
Enter text: bad text
Validation error: input starts with 'b' - not uppercase
Enter text: 21313
Validation error: input starts with '2' - not uppercase
Enter text: [][
Validation error: input starts with '[' - not uppercase
Enter text: F asdhfisuig
Enter text: Hello world
Enter text: /esc
```

Файл out.txt (были сохранены только валидные строки):

```
1   Good text
2   F asdhfisuig
3   Hello world
```

Ключевые элементы strace:

Создание каналов:

```
pipe2([4, 5], 0)      = 0
pipe2([6, 7], 0)      = 0
```

Создание клона (fork):

```
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f709f4bda10) = 7007
```

Закрытие ненужных каналов:

```
close(4)          = 0
close(7)          = 0
close(3)          = 0
```

Получение ввода из консоли:

```
read(0, "Good text\n", 1024)    = 10
```

Чтение ошибки из дескриптора дочернего процесса:

```
read(6, "input is empty\n", 1023) = 15
```

Вывод в консоль:

```
write(1, "Validation error: input is empty"..., 33) = 33
```

Ожидание завершения дочернего процесса:

```
wait4(-1, NULL, 0, NULL)       = 7007
```

Вывод: освоено создание дочерних процессов (fork), замена образа процесса (exec), настройка и использование каналов, предотвращение различных ошибок при выполнении. Произведен анализ системных вызовов в ходе работы программы с помощью strace.